

Chapter 11

Kubernetes in Action 1st Edition

Understanding K8S Internals

컨트롤 플레인 내부 동작 이해



신재근 (Jagger)

11장의 핵심 질문

`kubectl create deployment` 한 줄이 입력된 순간부터
Pod이 실제로 떴서 트래픽을 받기까지 — 내부에선 무슨 일이 일어나는가?

답하는 4가지 차원

- Architecture: 컨트롤 플레인을 이루는 컴포넌트들
- etcd & API Server: 어떻게 분산 일관성을 보장하나
- Controllers: 어떻게 협업하여 Pod을 만드나 (10단계 체인)
- High Availability: 어떻게 컨트롤 플레인 자체를 죽지 않게 만드나

시작 전 — 이 챕터를 보는 두 가지 관점

관점 1: K8s 운영자

디버깅 시 어디서 무슨 로그를 봐야 하는지 알게 됨

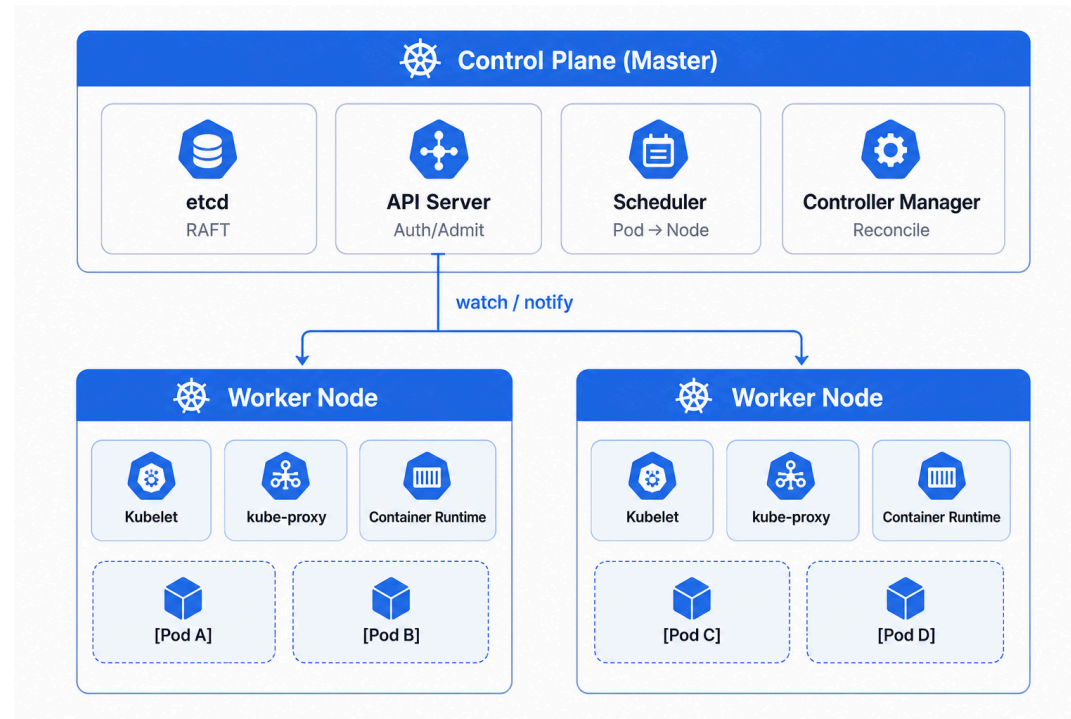
- Pod이 안 뜨면? Scheduler 로그 → Kubelet 로그 순서로 추적
- HA가 흔들리면? `kubectl get lease` 로 리더 교체 빈도 확인

관점 2: 분산 시스템 설계자

K8s가 풀어낸 분산 합의·일관성 문제 패턴 학습

- etcd의 RAFT — 합의 알고리즘 실전 적용
- API Server의 낙관적 락 — Stateless 서비스의 일관성 전략
- Lease 기반 리더 선출 — Active/Standby 패턴의 표준 구현

11.1 Architecture — 컴포넌트 구성



- **Control Plane:** etcd · API Server · Scheduler · Controller Manager
- **Worker Node:** Kubelet · kube-proxy · Container Runtime · Add-ons (DNS·CNI)

통신 규칙 — 단순 명료

핵심 원칙

모든 컴포넌트는 API Server하고만 통신한다.
etcd는 API Server하고만 통신한다.

의미

- 컴포넌트끼리 직접 통신 X → 느슨한 결합
- 새 컴포넌트 추가가 쉬움 — API Server만 알면 됨
- 보안 경계가 단순 — API Server만 잘 지키면 됨

예외 1가지

- `kubectl logs`, `kubectl attach`, `kubectl port-forward` 시
- → API Server → Kubelet 직접 연결 (역방향)

컴포넌트 H/A 전략 — 두 가지 패턴

컴포넌트	전략	이유
etcd	다중 활성 (3·5·7대 동시)	RAFT 분산 합의로 일관성 보장
API Server	다중 활성 (Stateless + LB)	모든 상태가 etcd에 있음 → 누가 처리해도 동일
Scheduler	Active/Standby (Lease 선출)	동시 실행 시 같은 Pod에 다른 Node 배정 위험
Controller Manager	Active/Standby (Lease 선출)	동시 실행 시 중복 reconcile 위험
Kubelet	노드당 1개 (시스템 데몬)	Pod 실행 주체 — 대체 불가

→ 컴포넌트의 stateful 여부와 동시 작업 가능 여부가 전략을 결정

컴포넌트 실행 방법

마스터 노드

- etcd · API Server · Controller Manager · Scheduler — Pod으로 실행 (Static Pod)
- 그래서 `kubectl get pods -n kube-system`으로 보임

워커 노드

- Kubelet만 시스템 데몬 (OS 직접 실행)
- 이유: Pod 실행의 주체 — 자기 자신은 Pod일 수 없음 (닭과 달걀)
- kube-proxy, CNI agent — Pod (DaemonSet)으로 실행

```
$ kubectl get po -o wide -n kube-system --sort-by spec.nodeName
NAME                                NODE
etcd-master                         master
kube-apiserver-master              master
kube-controller-manager-master     master
kube-scheduler-master              master
kube-proxy-xxx                     worker-1
```

11.1.2 etcd — 분산 KV 저장소

역할

클러스터의 모든 상태를 저장하는 단 하나의 진실 소스 (Single Source of Truth)

- Pod, Service, Deployment, ConfigMap... 모든 리소스 매니페스트
- 모든 컴포넌트의 desired state · status

왜 etcd인가

- 빠르고 일관된 분산 KV 저장소
- Watch API — 변경 알림을 클라이언트에 푸시
- 낙관적 동시성 제어 내장

접근 규칙

- API Server만 etcd에 접근 — 모든 락·검증을 한 곳에 집중

etcd 데이터 저장 구조

```
# 모든 K8s 리소스는 /registry 아래에 저장
$ etcdctl get /registry --prefix --keys-only
/registry/configmaps/...
/registry/deployments/default/kubia
/registry/pods/default/kubia-0
/registry/services/default/kubernetes
/registry/statefulsets/default/kubia

# 특정 리소스 조회 (kubectl --raw 추천)
$ kubectl get --raw /api/v1/namespaces/default/pods/kubia-0
```

두 가지 운영 메모

- etcd v3 부터 Protobuf로 저장 → `etcdctl get` 로 직접 보면 깨져 보임
- K8s 1.7부터 Secret은 etcd에 암호화 저장 (이전엔 평문이었음)

RAFT 합의 — 왜 etcd 인스턴스가 홀수인가

합의 알고리즘 핵심

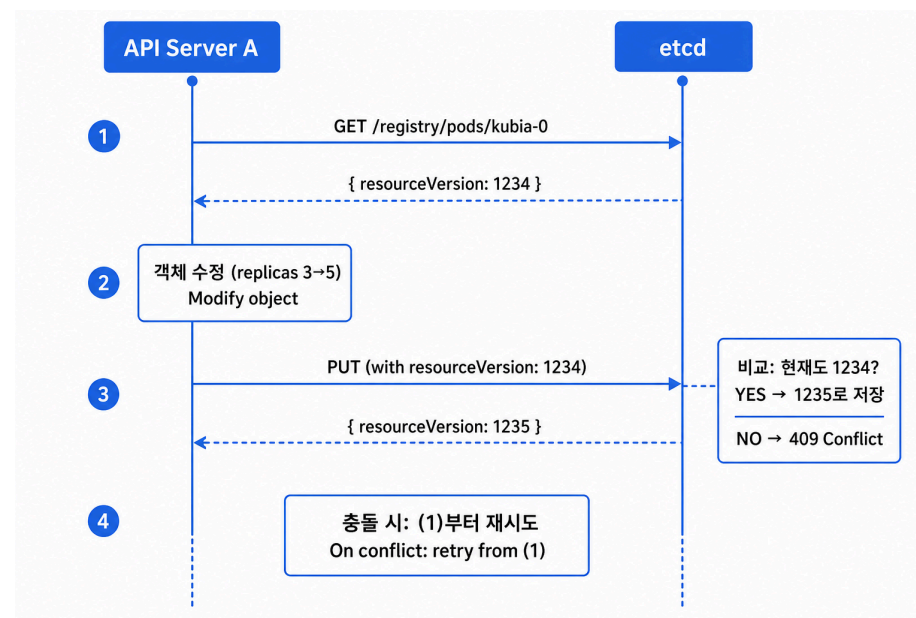
여러 노드 중 과반수가 동의한 상태만 valid로 인정

인스턴스 수	quorum (합의 필요 수)	허용 장애	비고
2	2	0	1대보다 더 나쁨 (장애 확률 2배)
3	2	1	가장 작은 HA 구성
4	3	1	3대와 동일 허용이지만 합의 부하↑
5	3	2	대규모 클러스터 권장
7	4	3	최대급 HA — 더 키우면 합의 비용↑

→ 짝수는 내결함성 이득 없이 합의 비용·네트워크 비용만 증가

낙관적 동시성 제어 (Optimistic Concurrency Control)

여러 API Server가 동시에 같은 객체를 업데이트할 때 일관성을 어떻게 보장하나



왜 단순 락이 아닌 낙관적 락인가:

- Stateless 다중 인스턴스 → 공유 락 불가, 충돌은 드물어 재시도가 효율적

낙관적 락이 K8s 내부에서 쓰이는 곳

사용자가 직접 마주치는 사례

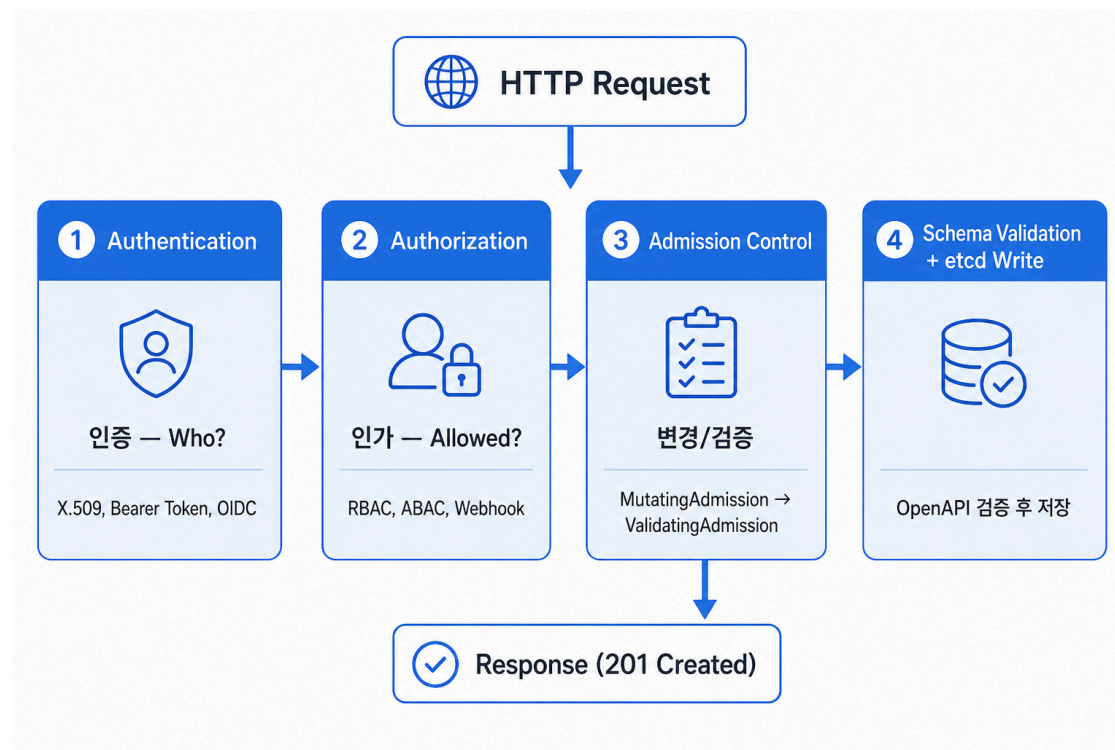
- `kubectl edit` 후 다른 사람이 이미 수정 → "the object has been modified" 에러
- `kubectl apply` 동시 실행 → 한쪽만 성공, 다른 쪽은 재시도

컨트롤러 내부

- 모든 Controller가 status 업데이트할 때 → 충돌 시 watch로 최신 받아 재시도
- HPA가 replicas 조정할 때 → 동시 수동 scale과 경합 시 재시도
- Scheduler가 Pod에 nodeName 배정할 때 → 다른 스케줄러와 경합 가능

낙관적 락은 K8s 전체의 일관성 모델을 떠받치는 기둥 — 화면에 안 보이지만 항상 작동

11.1.3 API Server — 4단계 파이프라인



→ 어느 단계든 거부되면 → 에러 응답, etcd 미반영

Admission Controller 주요 플러그인

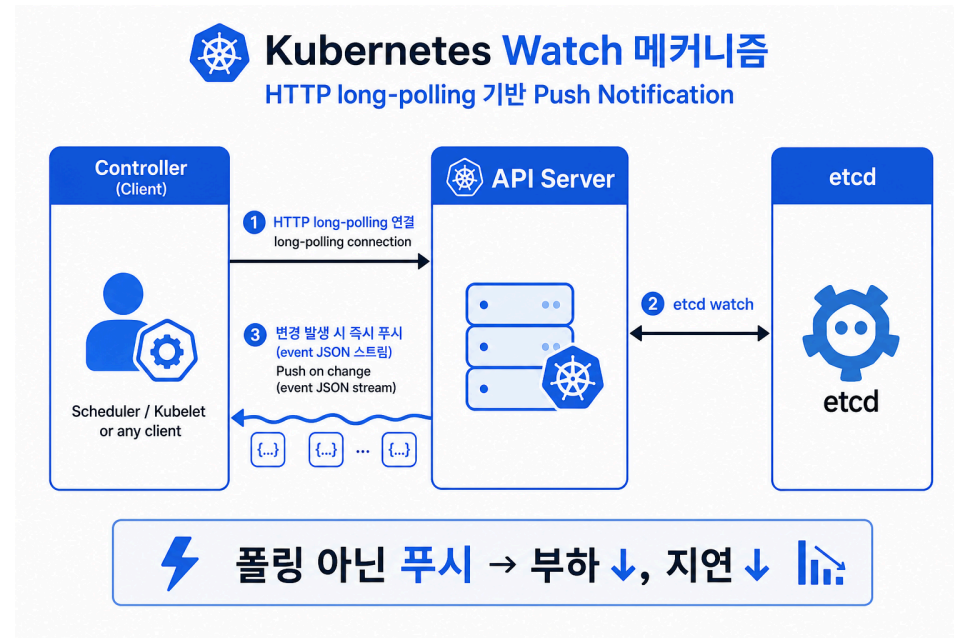
플러그인	동작	예시
AlwaysPullImages	imagePullPolicy를 강제 Always	이미지 보안 정책
ServiceAccount	미지정 Pod에 default SA 적용	거의 모든 Pod
NamespaceLifecycle	삭제 중·미존재 namespace에 Pod 생성 차단	namespace 정리 시
ResourceQuota	namespace CPU/메모리 할당량 초과 확인	멀티테넌트 환경
PodSecurityPolicy (deprecated → PSA)	privileged/hostPath 등 보안 정책 강제	1.25부터 PSA로 대체

Mutating vs Validating

- Mutating: 요청을 수정 (예: 누락 필드 default 채움)
- Validating: 요청을 검증만 (수정 X, 통과/거부만)
- 순서: Mutating → 스키마 검증 → Validating → etcd

Watch 메커니즘 — 폴링이 아닌 푸시

전통적: 클라이언트가 주기 폴링 → 부하·지연 트레이드오프. K8s는 다음과 같이 해결:



- HTTP long-polling으로 변경 알림 즉시 푸시 — 모든 컴포넌트가 이 메커니즘 사용

11.1.5 Scheduler — 두 단계 알고리즘

Scheduler의 일은 단순: 새 Pod의 `spec.nodeName` 을 결정해 API Server에 PATCH
실제 실행은 Kubelet이 — Scheduler는 배정만 함

단계 1: Filtering (배제)

현재 클러스터의 노드들 중 이 Pod을 받을 수 있는 노드만 필터링

단계 2: Prioritizing (점수)

살아남은 노드들에 점수를 매겨 최고 점수 노드 선택
동점 시 round-robin으로 분산

Filtering 단계 — 어떤 조건을 보나

체크 항목	질문
하드웨어 리소스	resource request 충족 가능?
리소스 압박	메모리·디스크 pressure 보고 중?
Node Selector	nodeSelector와 label 매칭?
Host Port	요청한 hostPort 이미 사용 중?
Volume 마운트	요청한 PV 마운트 가능? 다른 Pod이 이미 점유?
Taint/Toleration	Pod이 노드의 taint를 허용?
Affinity/Anti-Affinity	친화도 규칙 만족?

→ 모든 조건을 통과한 노드만 다음 단계(점수 계산)로

Prioritizing 단계 — 어떻게 점수 매기나

대표적 점수 함수들

- LeastRequestedPriority: CPU/메모리 여유 많은 노드 우대
- BalancedResourceAllocation: CPU·메모리 사용률 균형 잡힌 노드 우대
- NodeAffinityPriority: 친화도 가중치 적용
- InterPodAffinityPriority: 친한 Pod이 있는 노드 우대

동점 처리

- 동점이면 round-robin → 클러스터 전체에 Pod 고르게 분산

다중 스케줄러

- 여러 스케줄러 동시 실행 가능 — Pod의 `schedulerName` 필드로 지정
- 미지정 시 `default-scheduler` 사용

11.1.6 Controller Manager — 동작 모델

핵심 패턴: Reconciliation Loop

```
loop forever:
  current = API Server에서 실제 상태 조회
  desired = 리소스의 spec 섹션
  if current ≠ desired:
    API Server에 변경 요청
  status 섹션에 현재 상태 기록
```

모든 Controller에 공통

- API Server를 Watch → 변경 알림 받음
- 주기적 re-list → watch 이벤트 누락 대비 (안전망)
- 다른 Controller와 직접 통신 X → API Server를 통한 간접 협업

Controller끼리 통신 안 함이 K8s 확장성의 비결

Controller 종류 — 각자 자기 리소스만 본다

Controller	감시 리소스	생성 리소스
Replication Manager	ReplicationController	Pod
ReplicaSet/DaemonSet/Job	각 자기 리소스	Pod
Deployment	Deployment	ReplicaSet
StatefulSet	StatefulSet	Pod, PVC
Node	Node	(모니터링·연결 끊긴 노드 Pod 제거)
Service	Service (LB type)	외부 LB API 호출
Endpoint	Service + Pod	Endpoint
PV	PVC	PV 매칭

→ 각 Controller는 **단일 책임** — 자기 리소스만 감시·생성

11.1.7 Kubelet — 워커 노드의 모든 것

6가지 책임

1. Node 등록: 자기 노드를 Node 리소스로 API Server에 등록
2. Pod 모니터링: API Server를 watch하여 자기 노드 할당 Pod 감지
3. 컨테이너 실행: Container Runtime(containerd/Docker)에 실행 지시
4. 상태 보고: Pod의 상태·이벤트·리소스 사용량 API Server에 보고
5. Liveness Probe 실행: 실패 시 컨테이너 재시작
6. Pod 종료: API Server에서 삭제 감지 시 컨테이너 종료

Pod을 실제로 만드는 건 Kubelet 한 명 — 다른 컴포넌트는 모두 "지시"만

Static Pod — API Server 없이도 실행

메커니즘

- Kubelet이 로컬 디렉토리(`/etc/kubernetes/manifests`) 감시
- 거기에 Pod manifest 파일 두면 → Kubelet이 직접 실행

어디 쓰이나

- kubeadm 클러스터의 Control Plane 자체 — 닭과 달걀 문제 해결
- API Server가 죽어도 Static Pod은 계속 실행
- 부팅 순서: Kubelet → Static Pod (etcd, API Server) → 일반 Pod 가능

```
$ ls /etc/kubernetes/manifests/  
etcd.yaml  
kube-apiserver.yaml  
kube-controller-manager.yaml  
kube-scheduler.yaml
```

11.1.8 kube-proxy — Service의 실체

두 가지 모드

모드	동작	성능	LB 방식
userspace (초기)	kube-proxy가 실제 프록시 — 모든 패킷이 user space 경유	느림	Round-Robin (균등)
iptables (현재 기본)	kube-proxy는 iptables 규칙만 설정 — 패킷은 커널에서만 처리	빠름	Random (소수 시 불균등 가능)

iptables 모드 동작

1. Service 생성 → API Server가 가상 IP 할당
2. 모든 노드의 kube-proxy가 watch → iptables 규칙 추가
3. 클라이언트 패킷 (service IP:port) → 커널에서 DNAT → 실제 Pod IP:port로 변경 후 라우팅

→ Service IP는 ping 불가 (iptables 규칙일 뿐, 실제 IF 없음)

11.1.9 Add-ons — 핵심은 아니지만 거의 필수

Add-on	역할	동작 방식
CoreDNS	클러스터 내 DNS 조회	<code>kube-dns</code> Service로 노출, 모든 Pod의 <code>/etc/resolv.conf</code> 에 등록
Ingress Controller	외부 HTTP 라우팅	Ingress·Service·Endpoint watch → Nginx config 갱신
Dashboard	웹 UI	API Server를 호출하는 단순 클라이언트
CNI Plugin	Pod 네트워킹	각 노드에 DaemonSet으로 배포

흥미로운 사실

- Ingress Controller는 트래픽을 Service IP로 안 보냄 → Endpoint에서 Pod IP 직접 호출
- 이유: client IP 보존 (X-Forwarded-For 등)

11.2 Controllers의 협업 — 11장의 클라이맥스

지금까지 각 컴포넌트를 따로따로 봤습니다.

이제 이 모든 것이 어떻게 한 줄의 명령에 협력하는지 봅시다:

```
$ kubectl create deployment kubia --image=luksa/kubia:v1 --replicas=3
```

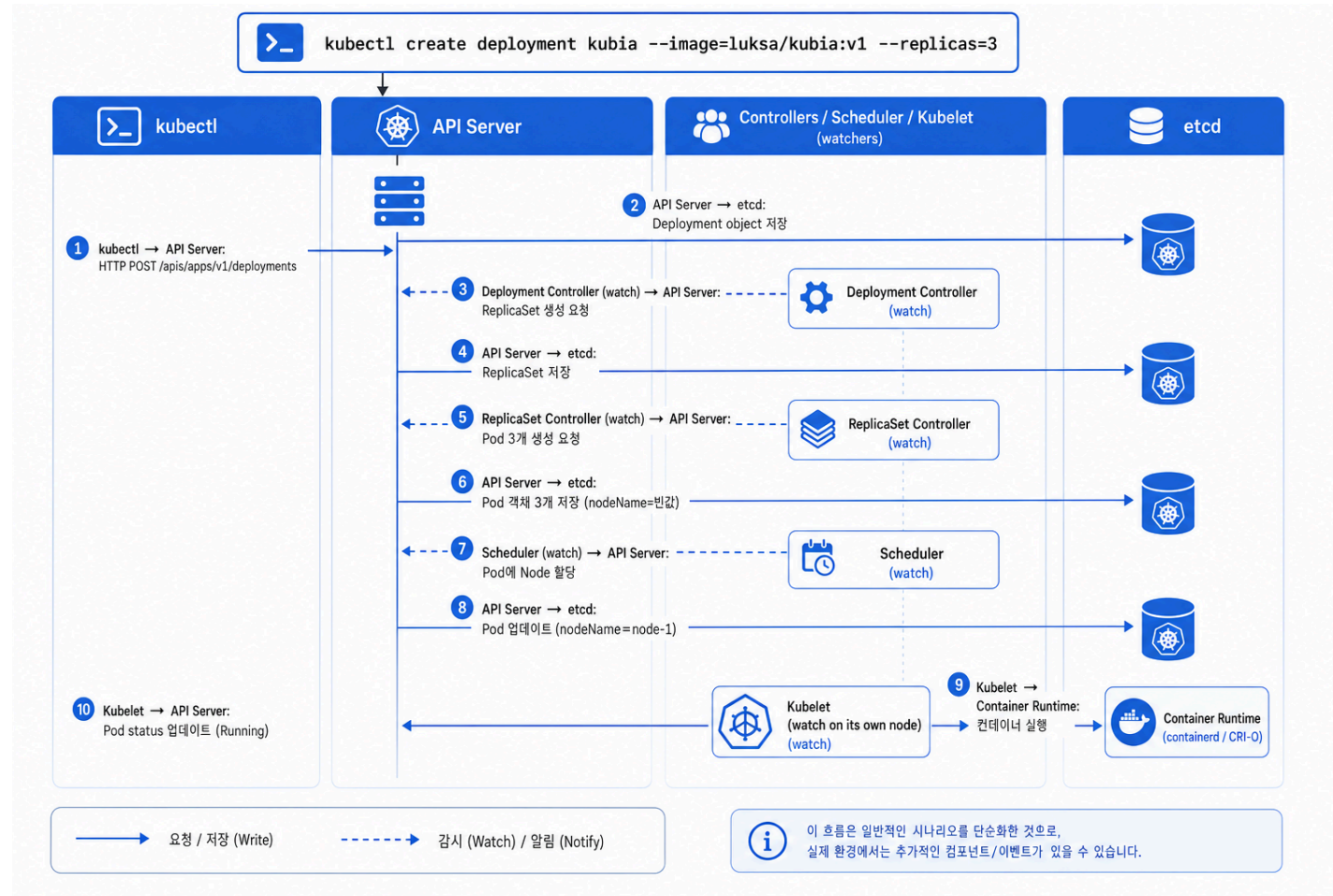
→ 이 한 줄로 Pod 3개가 떴서 트래픽을 받기까지

→ 10단계의 이벤트 체이닝이 일어남

핵심 통찰: 누구도 "직접" 명령하지 않음.

각자 API Server를 watch하다가, 자기 리소스가 변경되면 다음 리소스를 만들 뿐.

이벤트 체이닝 10단계 — 전체 흐름



이벤트 체이닝 — 핵심 관찰

- [7]단계까지 어떤 컨테이너도 실행되지 않음 — 모든 작업은 API 객체 CRUD만
- 모든 컴포넌트는 서로 직접 통신 X — API Server만 바라봄
- 이벤트 기반(watch) — 폴링이 아닌 즉시 푸시
- 한 컴포넌트가 죽어도 다른 컴포넌트는 영향 없음 (느슨한 결합)

K8s 아키텍처의 핵심 디자인 원칙 — 컴포넌트는 watch만 하고, 변경은 API Server를 통해서만.
이 패턴 덕분에 컴포넌트 추가/교체가 쉽고, 부분 장애가 전파되지 않음.

이벤트 체이닝 — 실측 명령어

```
# 한 터미널: 이벤트 watch 시작
$ kubectl get events --watch &

# 다른 터미널: Deployment 생성
$ kubectl create deployment kubia --image=luksa/kubia:v1 --replicas=3
```

기대되는 이벤트 시퀀스:

ScalingReplicaSet	Scaled up replica set kubia-xxx to 3
SuccessfulCreate	Created pod: kubia-xxx-yyy
Scheduled	Successfully assigned default/kubia-xxx-yyy to node-1
Pulled	Container image already present on machine
Created	Created container kubia
Started	Started container kubia

→ 각 줄이 10단계 시퀀스의 한 단계에 대응

11.3 실행 중인 Pod — Pause Container 미스터리

충격적 사실

단일 컨테이너 Pod에도 2개의 Docker 컨테이너가 실행됨

```
$ docker ps | grep kubia
nginx                "nginx -g 'daemon off'"    4s ago
gcr.io/.../pause:3.0  "/pause"                    7s ago    ← ★
```

Pause Container의 역할

- Pod의 모든 Linux namespace를 보유 (network, IPC, UTS)
- 모든 앱 컨테이너는 pause의 namespace를 공유
- 앱 컨테이너가 죽고 재시작되어도 동일 namespace에 참여 → IP 유지
- pause가 kill되면 Kubelet이 재생성 → 모든 앱 컨테이너도 재시작

Pause는 Pod의 인프라 — 보이지 않지만 모든 것을 떠받친다

11.4 Pod 간 네트워킹 — 요구 사항

3가지 요구 사항

1. 각 Pod은 고유한 IP — 클러스터 전체에서 유일
2. NAT 없는 flat 네트워크 — Pod A ↔ Pod B 패킷의 src/dst IP 보존
3. 외부망 접속 시는 노드 IP로 SNAT — 사설 IP 노출 방지

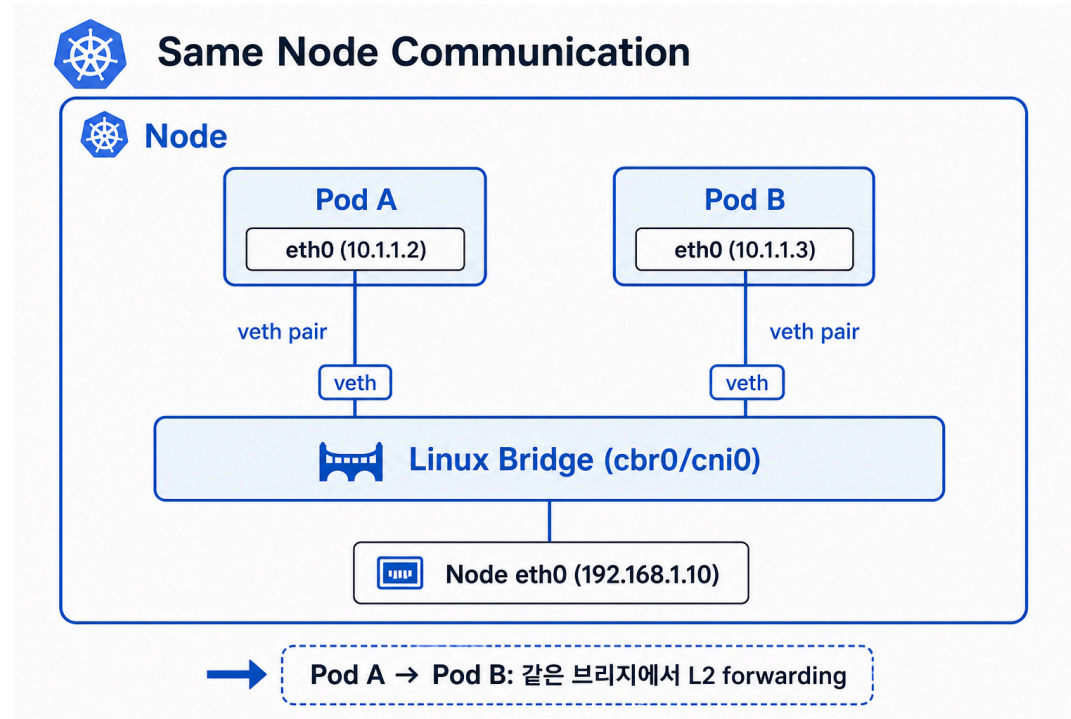
누가 이걸 구현하나

K8s 자체는 네트워킹을 직접 구현하지 않음 → CNI Plugin의 책임

대표 CNI:

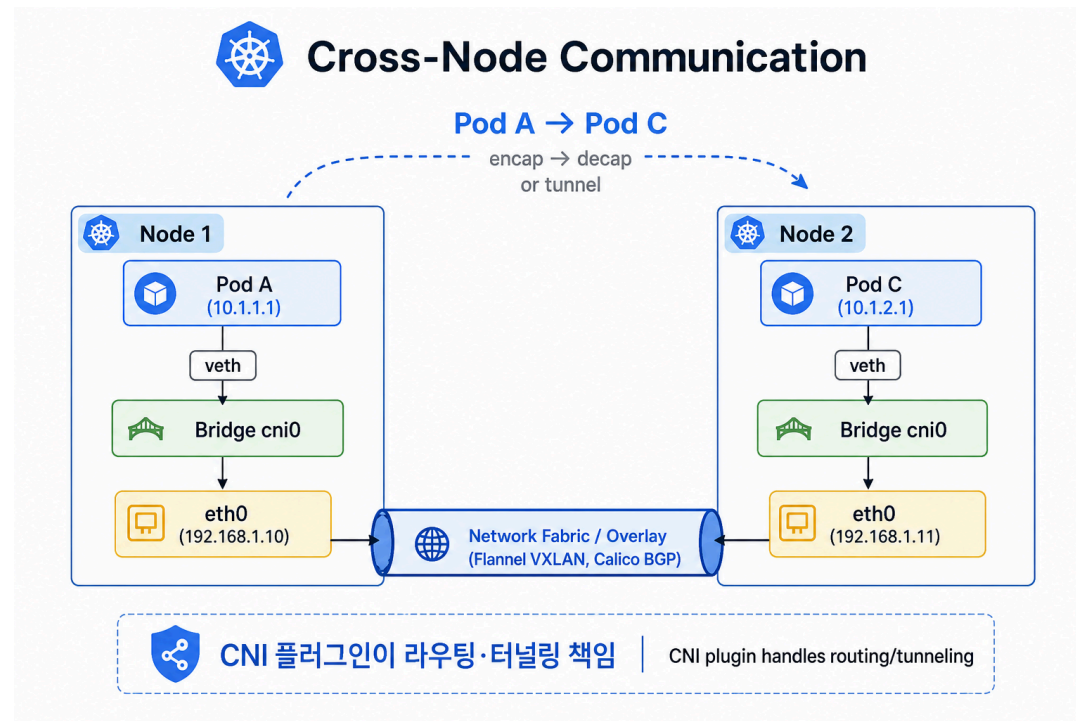
- Calico — BGP·iptables 기반, 정책·성능 강점
- Flannel — VXLAN 오버레이, 단순함
- Weave Net — 자동 디스커버리
- Cilium — eBPF 기반, 최신 트렌드

같은 노드 내 Pod 통신 — veth pair + bridge



- Pod 생성 시 veth pair 생성 → 호스트 bridge에 연결 → L2 직접 통신

다른 노드 간 Pod 통신 — CNI의 영역



- 각 노드의 bridge는 겹치지 않는 CIDR (Pod IP 충돌 방지) → CNI가 캡슐화/라우팅

11.5 Service의 실제 구현 — 가상 IP의 비밀

깜짝 사실

```
$ ping 10.96.0.1      # Service ClusterIP
PING 10.96.0.1: 56 data bytes
^C  # 응답 없음 (timeout)
```

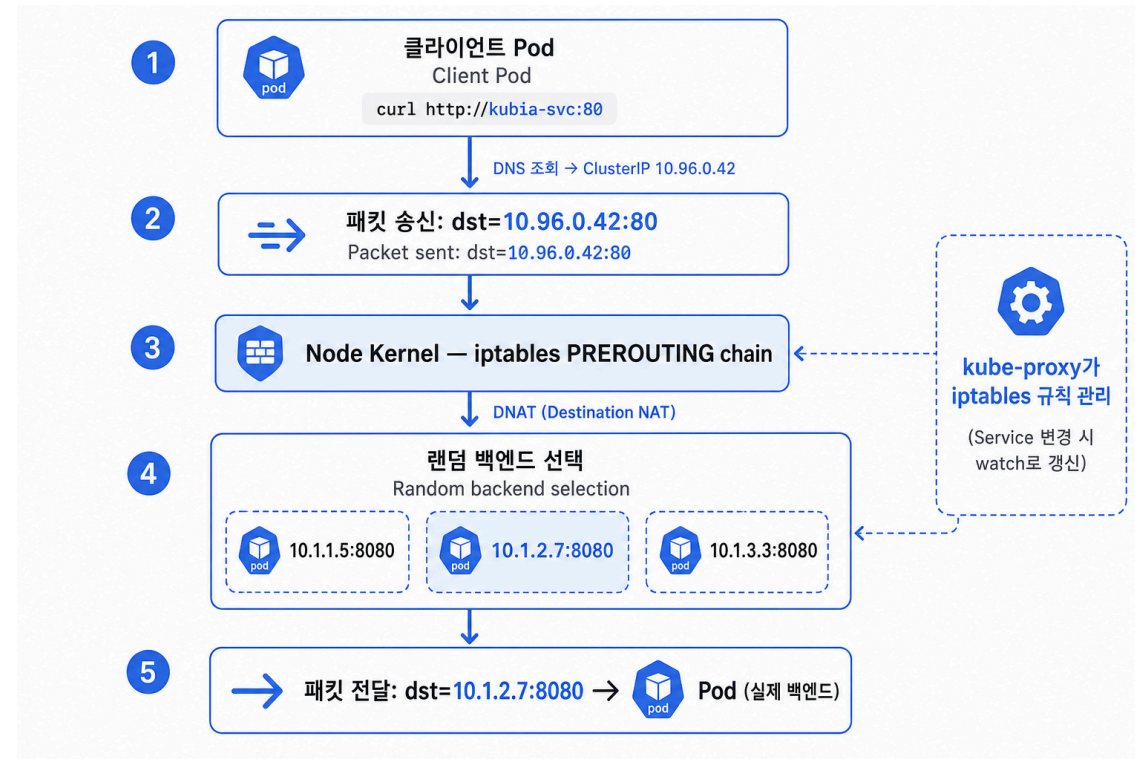
Service IP는 ping이 안 된다

이유

- Service IP는 가상 IP — 실제 네트워크 인터페이스에 할당 안 됨
- 단지 iptables 규칙에 등록된 표식일 뿐
- iptables가 "이 IP로 가는 패킷은 → Pod IP로 DNAT" 규칙만 보유
- ICMP는 DNAT 규칙이 없으니 응답 불가

Service는 IP+Port 쌍 — IP 단독으로는 의미 없음

Service IP → Pod IP 변환 흐름



- kube-proxy: iptables 규칙만 관리, 트래픽 처리 X (Service·Endpoint watch → 즉시 반영)

11.6 컨트롤 플레인 자체의 H/A

지금까지 본 모든 것은 Control Plane이 살아있다는 가정 하에 작동했습니다.

Control Plane이 죽으면?

- API Server 다운 → `kubectl` 자체가 안 됨
- Scheduler 다운 → 새 Pod이 노드에 배정 안 됨
- Controller Manager 다운 → Pod이 죽어도 재생성 안 됨
- etcd 다운 → 모든 것이 멈춤

그래서 Control Plane도 H/A로 운영해야 함

— 하지만 컴포넌트마다 전략이 다름

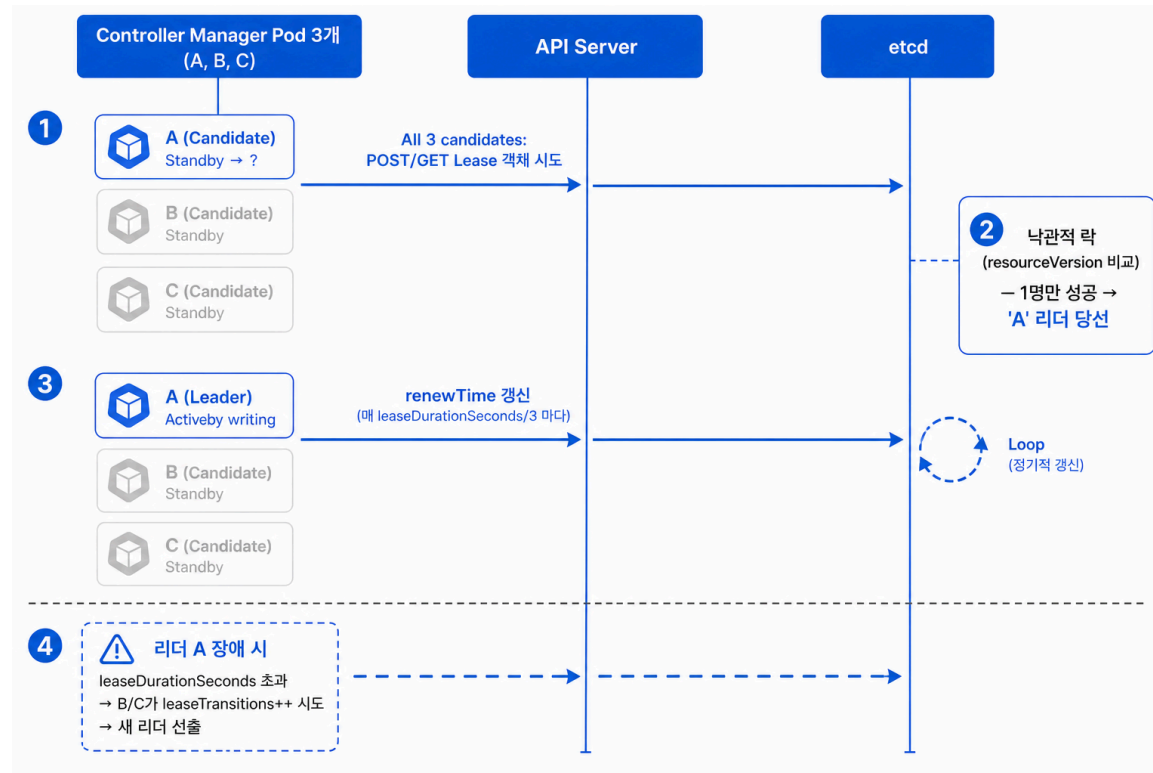
두 가지 H/A 전략 — 다시 보기

컴포넌트	전략	메커니즘
etcd	다중 활성 (모두 동시 작동)	RAFT 합의 — 분산 합의 알고리즘
API Server	다중 활성 (LB 뒤)	Stateless — 누가 처리해도 동일
Scheduler	Active/Standby (1개만)	Lease 기반 리더 선출
Controller Manager	Active/Standby (1개만)	Lease 기반 리더 선출

왜 컴포넌트마다 다른가

- etcd / API Server는 stateless 또는 분산 일관성 알고리즘 내장 → 동시 실행 안전
- Scheduler / Controller는 동시 실행 시 race condition 위험
 - 같은 Pod에 다른 Node 배정
 - 중복 reconcile (RS replicas: 3 → 6 만들어버림)

Lease 리더 선출 — 4단계 시퀀스



→ K8s 1.14+ 부터 `coordination.k8s.io/v1` Lease 객체 사용

Lease 객체 — 실측 YAML

```
$ kubectl get lease -n kube-system
NAME                                HOLDER                                AGE
kube-controller-manager            master1_23374c57-2886-48e8-ac56-bb1d460ef2a3    26d
kube-scheduler                     master1_ee7fbe2b-a477-4b9a-8230-f2acbeb365a1    26d

$ kubectl get lease kube-scheduler -n kube-system -o yaml
apiVersion: coordination.k8s.io/v1
kind: Lease
metadata:
  name: kube-scheduler
  namespace: kube-system
spec:
  acquireTime: "2023-06-08T08:01:26.690560Z" # 리더가 된 시점
  holderIdentity: master1_ee7fbe2b-...      # 현재 리더 식별자
  leaseDurationSeconds: 15                  # 갱신 안 되면 expire
  leaseTransitions: 1                       # 리더 교체 횟수
  renewTime: "2026-05-19T01:47:25.164855Z" # 마지막 갱신 시각
```

Lease 4개 핵심 필드 — 운영 활용

필드	의미	운영 활용
holderIdentity	현재 리더의 식별자 ({node}_{uuid})	"지금 일하는 게 누구야?"
leaseDurationSeconds	리스 유효 시간 (보통 15초)	너무 짧으면 false-positive · 너무 길면 fail-over 지연
renewTime	마지막 갱신 시간	"리더가 살아 있나?" 모니터링 지표
leaseTransitions	누적 리더 교체 횟수	자주 늘면 = 클러스터 불안정 (네트워크·etcd 부하)

운영 디버깅 팁

```
# 리더 교체 빈도 (값이 자주 늘면 문제)
kubectl get lease -n kube-system kube-controller-manager \
  -o jsonpath='{.spec.leaseTransitions}'

# 마지막 갱신 시점 (오래되면 모든 인스턴스 죽음)
kubectl get lease -n kube-system kube-scheduler \
  -o jsonpath='{.spec.renewTime}'
```

11장 핵심 패턴 정리 — 분산 시스템 설계 인사이트

패턴	어디서 쓰이나	일반 분산 시스템에서의 가치
단일 진실 소스	etcd가 모든 상태 보관	분산 동기화 복잡도 ↓
단일 진입점	API Server만 etcd 접근	락·검증·인증을 한 곳에 집중
이벤트 기반 (Watch)	모든 컴포넌트의 변경 감지	폴링 부하 ↓ · 지연 ↓
낙관적 락	resourceVersion 비교	Stateless 다중 인스턴스 가능
느슨한 결합	컴포넌트끼리 직접 통신 X	새 기능 추가·확장 쉬움
Reconciliation Loop	모든 Controller	Self-healing — 일시적 오류 자동 복구
Lease 리더 선출	Scheduler·Controller Manager	Active/Standby 표준 구현

→ K8s = 분산 시스템 설계 패턴의 종합 교과서

11장이 안 다룬 것 — 운영 시 추가 학습 포인트

보안

- 인증·인가 상세 — 12장에서
- NetworkPolicy — Pod 간 트래픽 제어
- PodSecurity — privileged·hostPath 제어

관찰 가능성

- Metrics Server — kubectl top
- Prometheus + Grafana — 메트릭 수집·시각화
- EFK Stack — 로그 수집

고급 운영

- Operator 패턴 — CRD + Custom Controller
- Cluster Autoscaler — 노드 자동 스케일
- Multi-Cluster — Federation, Karmada

→ 11장은 K8s 내부 이해의 시작 — 운영의 전부가 아님

11장 회고 — 무엇을 얻었나

Before 11장

"kubectl로 명령하면 K8s가 알아서 해준다" — 마법 박스로 보였음

After 11장

"10단계 이벤트 체이닝과 Watch·낙관적 락·Lease가 협업해서 Pod을 띄운다" — 명확한 메커니즘으로 보임

운영 디버깅 시 차이

- 이전: "Pod이 안 떠요" → 막막함
- 이후: Scheduler·Kubelet 로그 추적 → 원인 분석 가능

마법은 사라졌지만, 디버깅 능력은 생겼다

1판 vs 2판 — 11장의 운명

1판 (2017): "내부 이해" 단일 챕터

전체 컨트롤 플레인을 한 챕터에 담은 백과사전식

- 장점: 한 곳에서 조망 가능
- 단점: 실습 없이 길어 지루함

2판 (2024): **챕터 자체가 사라짐**

- etcd·API Server → 부록 또는 도입부
- 각 Controller → 해당 리소스 챕터에 분산
- Scheduling → 별도 챕터로 확장 (Topology Spread 등)
- Networking → 별도 네트워킹 챕터 (Gateway API 추가)

→ 2판은 학습 흐름 중심 — 11장의 백과사전식 모델은 폐기됨

Q & A

다음 챕터 미리보기

12장: API Server 보안 — 인증·인가·RBAC 상세

13장: Pod 보안 — SecurityContext·PSA